

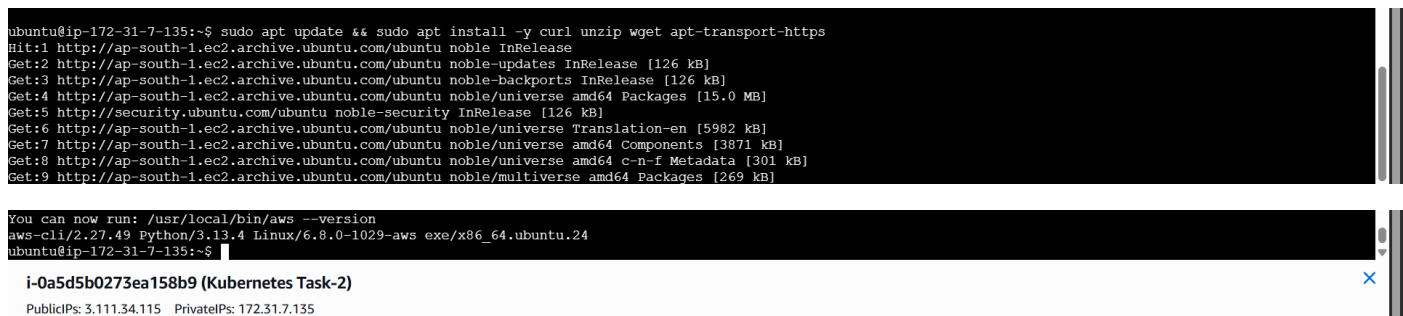
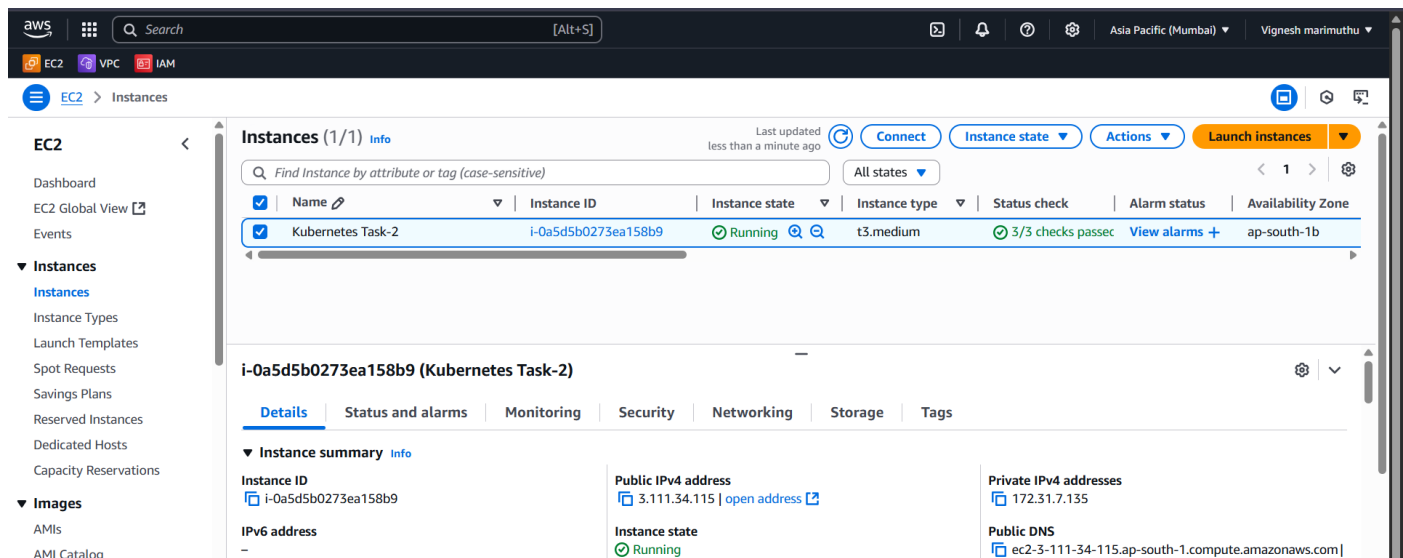
Kubernetes Task-2

Task Description:

Create the K8s EKS, further you have to do the deployment of the Nginx application and access the application outside the cluster.

Solution:

✓ 1. Pre-requisites on EC2 Ubuntu



- Installed dependencies
- EKS cluster creation and management require CLI tools like eksctl, kubectl, and aws-cli.
- Installing them on EC2 provides a secure, isolated workspace connected directly to AWS.
- AWS CLI lets EC2 talk to AWS services (like creating EKS).

✓ 2. Configure AWS CLI

```
ubuntu@ip-172-31-7-135:~$ aws configure
AWS Access Key ID [None]: AKIA3PC7TWY2VJBMPE7
AWS Secret Access Key [None]: l3pKlNg1CvpsSVMHlhBTGyLbfJisaz0ZC5JVgHEi
Default region name [None]: ap-south-1
Default output format [None]: json
ubuntu@ip-172-31-7-135:~$
```

i-0a5d5b0273ea158b9 (Kubernetes Task-2)

PublicIPs: 3.111.34.115 PrivateIPs: 172.31.7.135

- Authenticates the CLI with my AWS account using IAM credentials.

✓ 3. Install kubectl

```
ubuntu@ip-172-31-7-135:~$ kubectl version --client
Client Version: v1.33.2
Kustomize Version: v5.6.0
ubuntu@ip-172-31-7-135:~$
```

i-0a5d5b0273ea158b9 (Kubernetes Task-2)

PublicIPs: 3.111.34.115 PrivateIPs: 172.31.7.135

- kubectl manages and interacts with the EKS cluster.

✓ 4. Install eksctl

```
eksctl version
0.210.0
ubuntu@ip-172-31-7-135:~$
```

i-0a5d5b0273ea158b9 (Kubernetes Task-2)

PublicIPs: 3.111.34.115 PrivateIPs: 172.31.7.135

- eksctl is a CLI to create and manage EKS clusters easily.

✓ 5. Create EKS cluster with eksctl

```
ubuntu@ip-172-31-7-135:~$ eksctl create cluster --name cluster1 --region ap-south-1 --nodegroup-name nodegroup1 --node-type t3.medium --nodes 1
2025-07-09 16:45:26 [i] eksctl version 0.210.0
2025-07-09 16:45:26 [i] using region ap-south-1
2025-07-09 16:45:26 [i] setting availability zones to [ap-south-1c ap-south-1a ap-south-1b]
2025-07-09 16:45:26 [i] subnets for ap-south-1c - public:192.168.0.0/19 private:192.168.96.0/19
2025-07-09 16:45:26 [i] subnets for ap-south-1a - public:192.168.32.0/19 private:192.168.128.0/19
2025-07-09 16:45:26 [i] subnets for ap-south-1b - public:192.168.64.0/19 private:192.168.160.0/19
```

```
ubuntu@ip-172-31-7-135:~$ eksctl get cluster --region ap-south-1
NAME      REGION    EKSCCTL CREATED
cluster1  ap-south-1  True
ubuntu@ip-172-31-7-135:~$
```

i-0a5d5b0273ea158b9 (Kubernetes Task-2)

PublicIPs: 3.111.34.115 PrivateIPs: 172.31.7.135

✓ 6. Create Deployment

```
Kustomize Version: v5.6.0
ubuntu@ip-172-31-7-135:~$ kubectl create deployment nginx-deploy --image=nginx
deployment.apps/nginx-deploy created
ubuntu@ip-172-31-7-135:~$
```

i-0a5d5b0273ea158b9 (Kubernetes Task-2)

PublicIPs: 3.111.34.115 PrivateIPs: 172.31.7.135

- Creates a deployment named nginx-deploy using the official nginx Docker image.

✓ 7. Verify deployment

```
ubuntu@ip-172-31-7-135:~$ kubectl get deployments
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
nginx-deploy  1/1     1             1           29s
ubuntu@ip-172-31-7-135:~$
```

i-0a5d5b0273ea158b9 (Kubernetes Task-2)

PublicIPs: 3.111.34.115 PrivateIPs: 172.31.7.135

✓ 8. Expose as LoadBalancer service

```
ubuntu@ip-172-31-7-135:~$ kubectl expose deployment nginx-deploy --port=80 --type=LoadBalancer
service/nginx-deploy exposed
ubuntu@ip-172-31-7-135:~$
```

i-0a5d5b0273ea158b9 (Kubernetes Task-2)

PublicIPs: 3.111.34.115 PrivateIPs: 172.31.7.135

- It creates a service that assigns an external IP from AWS ELB.

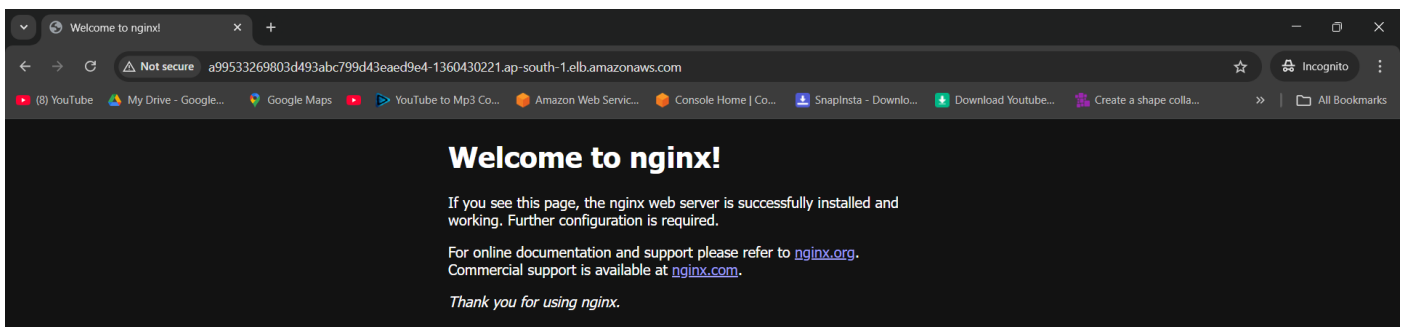
✓ 9. Get external URL

```
ubuntu@ip-172-31-7-135:~$ kubectl get services
NAME          TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
kubernetes    ClusterIP     10.100.0.1     <none>          443/TCP          12m
nginx-deploy   LoadBalancer 10.100.225.2   a99533269803d493abc799d43eae9e4-1360430221.ap-south-1.elb.amazonaws.com 80:32677/TCP    15s
```

i-0a5d5b0273ea158b9 (Kubernetes Task-2)

PublicIPs: 3.111.34.115 PrivateIPs: 172.31.7.135

✓ 10. Checking URL



Steps I Performed

- I launched an EC2 Ubuntu instance and installed the necessary tools including AWS CLI, kubectl, and eksctl.
- I used aws configure to authenticate my CLI with AWS IAM credentials.
- Then I created an EKS cluster using eksctl with two managed nodes.
- Once the cluster was ready, I used kubectl to deploy an Nginx application and exposed it using a LoadBalancer service.
- I retrieved the external IP provided by AWS ELB and accessed the Nginx welcome page from my browser.
- This demonstrated successful deployment and external access to an application on EKS.